

Lab Procedure for Python

Sensor Fusion

Setup

1. It is recommended that you review [Lab 2 – Application Guide](#) before starting.
2. Turn on the QBot Platform by pressing the power button once. To ensure the robot is ready for the lab, check the following conditions:
 - a. Confirm that LEDs on the robot base are solid red, and the LCD shows a battery voltage above 12.5 V.
 - b. The Logitech F710 joystick's wireless receiver is connected to the QBot Platform. Ensure the switch on the joystick is in the **X position** and that the LED beside the Mode button is **off**.
 - c. Connect your computer to the same network that the QBot Platform is connected to. If using the provided router, connect to the Wi-Fi network **Quanser_UVS-5G**.

3. Test connectivity: Open a command prompt on your local computer, enter the following command, replacing `192.168.x.x` with the QBot's IP address found on its LCD display, and hit enter: `ping 192.168.x.x`

A successful connection returns a line such as: `Reply from 192.168.x.x: bytes=32 time=20ms TTL=64.`

4. In Section A of [sensor_fusion.py](#), set **ipHost** to your Windows computer's IPv4 address. To find it, run `ipconfig` in Command Prompt and look under your active network connection. Your IP address is listed as the IPv4 address, and should look like the one for the QBot, `192.168.X.X`.
5. The code for this lab builds on **Lab 1: Localization** in Skills Progression 2. Ensure that the relevant code sections are completed with your code from Lab 1:
 - a. In Section B, right-click on `QBPRanging()` and select "go to definition" to view the class definition within the [qbot_platform_functions](#) module. Complete the functions `adjust_and_subsample()` and `correct_lidar()`. In the same file, scroll to the `QBPLocalization` class and complete the functions `save_ref()` and `scan_match()`.
 - b. In [sensor_fusion.py](#), complete the code in Sections D and E to read and correct raw LiDAR data, save a reference scan, and set the **refCollected** flag.

- c. Modify the line that calls the `scan_match()` function to pass in the corresponding parameter values from Lab 1.

Running Code

You will edit the code on your local Windows machine and then transfer it to the QBot Platform. Repeat the steps in this section each time you change the code. **Refer back to this section later when instructed to run the code.**

1. Open WinSCP, use the QBot's IP address as the **Host name**, and "nvidia" for both Username and Password. Select Login. This opens a connection to the QBot for file transfer.
2. In the WinSCP window, transfer the following files from your local Windows machine to the QBot:
 - a. On the QBot, open
/home/nvidia/Documents/Quanser/6_teaching/3_Robotics/sp2_surveying/l2_sensor_fusion/hardware/python shown in Figure 1, then copy the updated `sensor_fusion.py` and `qbot_platform_driver_physical.rt-linux_qbot_platform` into that directory.

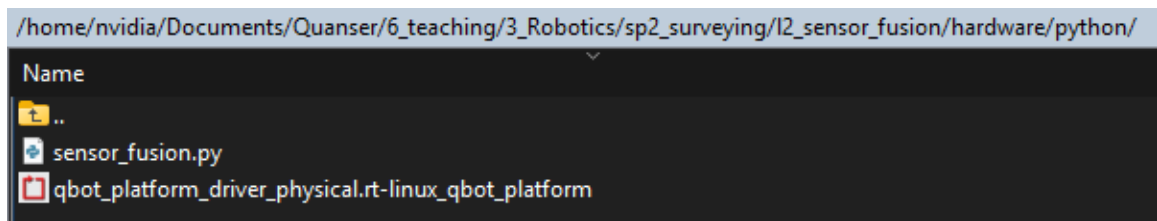


Figure 1. QBot directory for python script

- b. Navigate to the directory
/home/nvidia/Documents/Quanser/0_libraries/python/ shown in Figure 2, or create these directories if they do not exist. Copy over the "hal" and "pal" folder from your local Windows machine, replacing existing files on QBot.

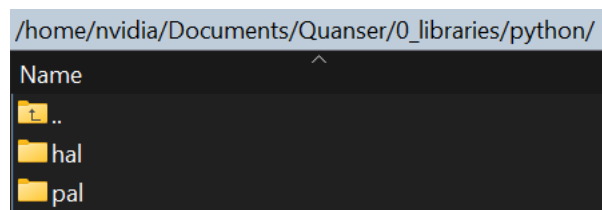


Figure 2: 'hal' and 'pal' folder in correct directory

3. Run [observer.py](#) on your local Windows machine first to initiate receiving data feeds from the QBot Platform. Keep this script running on your Windows machine while you run [sensor_fusion.py](#) in the next step.
4. Run [sensor_fusion.py](#) on QBot Platform:
 - a. Open a PuTTY session: at the top menu bar, click Commands > Open in PuTTY.
 - b. In the PuTTY terminal, enter the password "nvidia".
 - c. Navigate to the python script directory from step 2a by using the `cd` command.
 - d. Run the script using the following command:

```
sudo PYTHONPATH=$PYTHONPATH python3 sensor_fusion.py
```

The LEDs on the base of the QBot will turn blue when the scripts starts successfully.
5. To stop both [sensor_fusion.py](#) and [observer.py](#), press the right button (RB) on the joystick.

Estimating Forward Speed

1. In this lab you will be exploring sensor fusion techniques to localize the QBot Platform, beginning with estimating the forward speed of the QBot.
2. View the class definition for **QBPMovement()** in Section B within the [qbot_platform_functions](#) module. Scroll down to the function **diff_drive_forward_velocity_kinematics()** and complete the function by copying the relevant code section over from the **Forward Kinematics** lab in Skills Progression 1. This function calculates **fwdSpdOdom** and **turnSpdOdom**, which are the odometry-based estimates of the forward and turn speeds, respectively.
3. Next, we will complete the code to estimate the forward speed using the accelerometer.

To drive the QBot around using the joystick controller, press and hold the left button (LB) to arm the QBot, and keep LB pressed as you drive the QBot using the left and right joysticks. Note that this lab does not include obstacle detection functionality. To stop the QBot, disarm it by releasing LB. The joystick controls are as follows:

- a. Left joystick: Move left or right to turn the QBot body
- b. Right joystick: Move up or down to drive the QBot forward or back
- c. A button: (optional) Drive the QBot in a circle by applying a constant forward and turning speed command.

Run the code. Drive the QBot around while observing the X, Y, and Z signals on the Acceleration scope. Which direction corresponds to the forward movement direction? Stop the code.

4. In Section C, modify the line **fwdAccel = 0** to set the variable **fwdAccel** to the forward acceleration component you identified in Step 3. This value is passed to the **accelIntegrator** generator, which accumulates a running sum of **fwdAccel*dt** at each loop to produce the integrated forward velocity.
5. Run the code but do not drive the QBot around. Instead, let the QBot remain stationary for at least one minute and observe the Forward Speed scope, which displays the individual estimates from each sensor. Then, drive the QBot forward and backward.
6. What do you observe about the accelerometer-based forward speed estimate when the QBot is stationary? Take a screenshot to show your observations. Why does this happen? Stop the code.
7. In Section F, modify the line **'Forward Speed Estimate': [fwdSpdAccel, 0, 0]** to replace the first **0** with **fwdSpdOdom**. This plots the odometry-based estimate on the Forward Speed scope. Repeat Step 5 while observing both signals on the scope. What do you observe about the forward speed estimate based on the wheel speeds? How does this estimate compare to the accelerometer-based estimate? Take a screenshot to show your observations.
8. Stop your code. Now, we will correct the error in the accelerometer-based estimate using only proportional control. Right-click on the function **complementary_filter()** in Section B and view its definition. When this function is called in the main code, in Section C, the forward acceleration (**fwdAccel**) and the encoder-based speed estimate (**fwdSpdOdom**) are passed in as the **rate** and **correction** values. Here, the **error** is calculated as the difference between the generator **output** and the **correction** value passed into the generator. The error, multiplied by a proportional gain **Kp**, is subtracted from the **rate**. The result is integrated and becomes the output of the generator.
9. The line **fwdSpdFused = fwdSpeedComplementaryFilter.send((fwdAccel, fwdSpdOdom, 0.5, 0.0))** in Section C sends **fwdAccel** and **fwdSpdOdom** to the generator **fwdSpeedComplementaryFilter** with gains **Kp** set to 0.5 and **Ki** set to 0. In the line **'Forward Speed Estimate': [fwdSpdAccel, fwdSpdOdom, 0]** in Section F, replace the **0** with **fwdSpdFused** to plot the fused estimate on the Forward Speed Scope.
10. Run the code and drive the QBot around while observing the Forward Speed Scope. Stop the code.

11. Pass in a value of 1.0 for K_p and repeat the previous step. How does the fused estimate compare to the estimates from each individual sensor? How does increasing K_p affect the fused estimate?
12. Now, add integral control to complete the complementary filter: go to the function definition for **complementary_filter()**. Modify the line beginning with **temp =** to subtract the integrated error value multiplied by gain K_i . Save your changes.
13. In [sensor_fusion.py](#), change K_i to 0.05 and run the code. Let the QBot idle for a minute and then drive it forward and backward. Stop the code. Explain why the integral controller was added.
14. Run your code and observe the signals as you tune the gains K_p and K_i . How does each value affect how the signals are fused together? Tune the gains until the fused forward speed accurately tracks the correction signal with minimal noise. A sample tuned signal is shown in Figure 3.

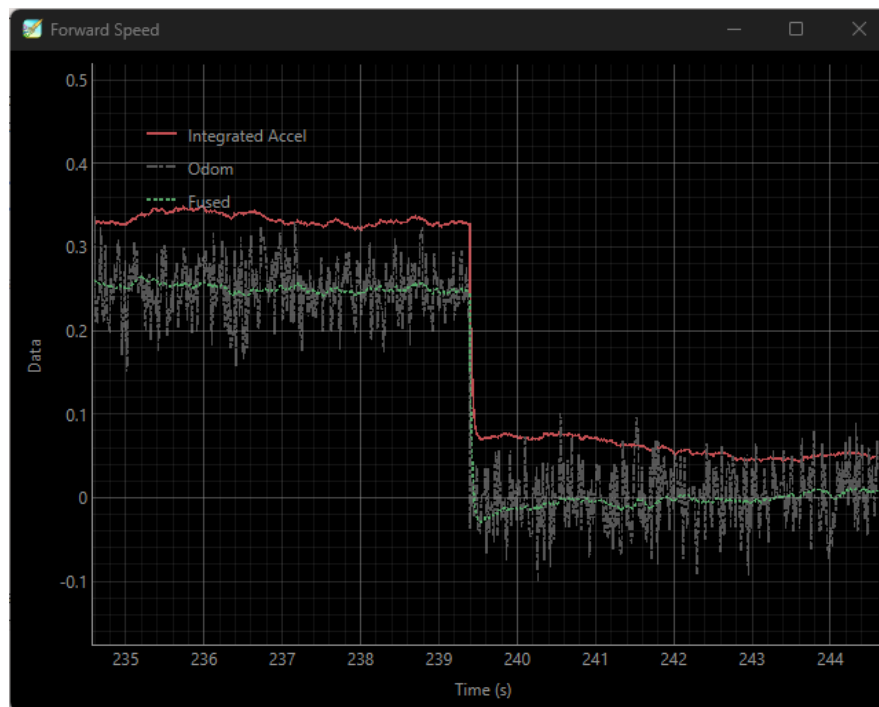


Figure 3: Fused forward speed estimate with tuned K_p and K_i gains

15. Complete the tuning for the two other instances of the complementary filter in Section C. Start with the **turnSpdComplementaryFilter**, which outputs the fused estimate of turn speed by fusing the rate of angular velocity about the z axis measured by the gyroscope, **gyroRate**, with an odometry-based estimate, **turnSpdOdom**. Observe the Turn Speed scope and tune the K_p and K_i gains passed to **turnSpdComplementaryFilter**.

16. You now have fused estimates of the QBot's forward speed and turn speed. These are converted to speeds along the X and Y directions and combined into a vector **[x speed (m/s), y speed (m/s), ω (rad/s)]**. The **poseComplementaryFilter** uses this as the rate input and estimates the current position of the QBot using the position estimate from scan match as the correction input, which is the vector **[x position (m), y position (m), theta (rad)]**.

Unlike the previous two filters, the K_p and K_i gains passed to **poseComplementaryFilter** are 3-element arrays. Each element sets the gain for the corresponding element of the pose vector. Tune each element using the X Position Estimates, Y Position Estimates, and Theta Estimates scopes to view the rate (integrated), correction, and fused estimates.

17. Run your code, drive the QBot around, and observe the estimated position on the Estimated Pose XY scope.

Comparison to Kalman Filter

18. Now, we will test an Extended Kalman Filter (EKF) for pose estimation and compare this sensor fusion technique to the complementary filter. The EKF takes the fused forward and turn speeds, `fwdSpdFused` and `turnSpdFused`, as control inputs. It uses them to predict the QBot's pose with a kinematic motion model, then corrects the prediction with the scan-match pose. The weight given to each correction is a gain calculated at every step to minimize the estimate's uncertainty, rather than set manually as in the complementary filter. This gain depends on the process noise covariance Q and the measurement noise covariance R , set by **Q_ekf** and **R_ekf** when **QBotEKF** is initialized in Section B. The completed implementation can be viewed inside the class definition for **QBotEKF()**.
19. In Section E, uncomment the line **poseEKF = ekf.x_hat[:,0]** to set poseEKF as the EKF position estimate. Run the code. Drive the QBot in a circle while observing the Pose Estimates Scope. How do the estimates from the EKF compare to the estimates from the complementary filter?
20. Turn OFF the QBot by single pressing the power button. Do not hold it down. After shutdown, confirm that all the LEDs are off.